

PyTorch

Interview-focused notes for ML placements, AI engineer roles, research internships, and production projects.

1. Overview

PyTorch is an open-source tensor-computation and deep learning framework. It provides:

- **Tensors** for fast numerical computation on CPUs, GPUs, and other accelerators.
- **Automatic differentiation** through `torch.autograd`.
- **Neural-network building blocks** through `torch.nn`.
- **Optimization algorithms** through `torch.optim`.
- **Data pipelines** through `Dataset`, `DataLoader`, samplers, and transforms.
- **Performance tools** such as automatic mixed precision, graph compilation, distributed training, profiling, and quantization.
- **Domain libraries** such as `torchvision`, `torchaudio`, and ecosystem integrations for Transformers, graph learning, reinforcement learning, and scientific computing.

PyTorch is not itself a neural-network algorithm. It is the software system used to express tensor programs, differentiate them, train models, evaluate them, and run inference.

Why PyTorch is useful

PyTorch combines a Python-friendly programming model with accelerator-backed kernels. A model's forward pass is usually ordinary Python code. While that code runs, autograd records differentiable tensor operations in a dynamic computation graph. Calling `loss.backward()` applies the chain rule and populates parameter gradients.

This makes PyTorch useful for both research and engineering:

Setting	Why PyTorch fits
Research	Dynamic graphs, easy debugging, custom operators, rapid model changes
Computer vision	Convolutions, pretrained backbones, image transforms, detection/segmentation tooling
NLP and LLMs	Embeddings, attention, distributed training, mixed precision, Hugging Face integration
Recommendation	Large embeddings, sparse gradients, distributed data/model parallelism
Generative AI	GANs, diffusion models, autoregressive models, custom sampling loops
Production inference	Checkpoint loading, compilation, serving integration, export paths
Scientific ML	Differentiable simulations, GPU tensor algebra, custom objectives

PyTorch's main abstraction layers

```
Application / experiment
    ↓
Training loop, evaluation, checkpointing
    ↓
nn.Module, losses, optimizers, schedulers
    ↓
Autograd computation graph
    ↓
Tensor operations and device kernels
    ↓
CPU / CUDA GPU / other supported accelerator
```

An interview-ready definition is:

PyTorch is a tensor and automatic-differentiation framework in which models are `nn.Module` objects, trainable values are `nn.Parameter` tensors, forward operations build an autograd graph, and optimizers update parameters using gradients produced by backpropagation.

2. Intuition

Think of PyTorch as a spreadsheet whose cells can live on a GPU and remember how they were calculated.

Suppose a prediction is:

```
y_hat = xw + b
loss = (y_hat - y)^2
```

If `w` and `b` require gradients, PyTorch records that multiplication, addition, subtraction, and square created `loss`. When `loss.backward()` is called, PyTorch walks backward through those recorded operations and computes:

```
d(loss)/dw and d(loss)/db
```

The optimizer then changes `w` and `b` slightly in the direction that should reduce the loss.

```
import torch

x = torch.tensor(3.0)
y = torch.tensor(7.0)
w = torch.tensor(1.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

y_hat = x * w + b
loss = (y_hat - y).pow(2)
```

```
loss.backward()

print(w.grad) # d(loss)/dw = 2 * (y_hat - y) * x = -24
print(b.grad) # d(loss)/db = 2 * (y_hat - y) = -8
```

Three ideas explain most of PyTorch:

1. **A tensor is data plus metadata** such as shape, dtype, device, strides, and possibly gradient history.
2. **A module is a stateful function** containing parameters, buffers, and child modules.
3. **Training is a repeated protocol**: load batch → forward → compute loss → clear gradients → backward → update parameters → evaluate.

The framework does not decide the model, objective, metrics, or experimental design. It gives the machinery; the engineer must still prevent leakage, choose suitable losses, validate correctly, and monitor generalization.

3. Prerequisites

Programming prerequisites

- Python functions, classes, iterators, context managers, and exceptions.
- NumPy arrays, indexing, vectorization, broadcasting, and shape manipulation.
- Basic debugging: inspecting types, shapes, devices, dtypes, and stack traces.
- Familiarity with packages, environments, and reproducible scripts.

Mathematical prerequisites

- Scalars, vectors, matrices, and higher-order tensors.
- Matrix multiplication and dot products.
- Derivatives, partial derivatives, gradients, and the chain rule.
- Probability distributions, likelihood, logarithms, expectation, and variance.
- Basic optimization: gradient descent, learning rate, momentum.

Machine-learning prerequisites

- Features, labels, model parameters, and hyperparameters.
- Train/validation/test splits.
- Loss functions versus evaluation metrics.
- Overfitting, regularization, class imbalance, and data leakage.
- Feed-forward networks, activations, and backpropagation.

Helpful systems knowledge

- CPU versus GPU memory.
 - Host-to-device transfer.
 - Parallel data loading.
 - Floating-point precision (`float32`, `float16`, `bfloat16`).
 - Throughput, latency, memory bandwidth, and synchronization.
-

4. Core Concepts

4.1 Tensors

A `torch.Tensor` is an n-dimensional, homogeneous array. All elements have one dtype and live on one device.

```
import torch

scalar = torch.tensor(4.0)           # shape: []
vector = torch.tensor([1.0, 2.0, 3.0]) # shape: [3]
matrix = torch.zeros(2, 3)           # shape: [2, 3]
image_batch = torch.randn(32, 3, 224, 224) # NCHW
```

Important tensor metadata:

Property	Meaning	Interview angle
<code>shape</code> / <code>size()</code>	Length of every dimension	Most runtime errors are shape errors
<code>dtype</code>	Numeric representation	Accuracy, memory, compatible operations
<code>device</code>	CPU or accelerator location	Inputs and parameters must normally match
<code>requires_grad</code>	Whether autograd tracks operations	Needed for learnable floating-point tensors
<code>grad</code>	Accumulated derivative for a leaf tensor	Must clear between updates
<code>stride()</code>	Step in storage for each dimension	Explains views and contiguity

Creation and conversion

```
x = torch.arange(12, dtype=torch.float32).reshape(3, 4)
same_storage_when_possible = torch.as_tensor([[1, 2], [3, 4]])
copied_tensor = torch.tensor([1, 2, 3])
random_normal = torch.randn(4, 8)
random_uniform = torch.rand(4, 8)
integers = torch.randint(0, 10, (5,))
like_x = torch.zeros_like(x)
```

`torch.tensor(existing_data)` generally constructs a new tensor. `torch.as_tensor(data)` avoids a copy when representation and device permit. `torch.from_numpy(array)` shares CPU memory with a compatible NumPy array, so mutation can be visible from both sides.

Shapes and dimension operations

```
x = torch.randn(8, 3, 32, 32)
```

```
flat = x.flatten(start_dim=1)      # [8, 3072]
channels_last_view = x.permute(0, 2, 3, 1) # [8, 32, 32, 3]
one_image = x[0]                  # [3, 32, 32]
kept_batch = x[0:1]               # [1, 3, 32, 32]
added_dim = one_image.unsqueeze(0) # [1, 3, 32, 32]
removed_dim = added_dim.squeeze(0) # [3, 32, 32]
```

Interview trap: `x[0]` removes a dimension, while `x[0:1]` preserves it.

Broadcasting

PyTorch compares dimensions from right to left. Two dimensions are compatible when they are equal or one of them is `1`; a missing leading dimension behaves like `1`.

```
batch = torch.randn(64, 10)
bias = torch.randn(10)
output = batch + bias # bias behaves like [1, 10]
```

Broadcasting avoids explicit copies, but a conceptually expanded result can still make a later operation expensive. Always verify that a broadcast is intentional; `[B, 1] + [B]` produces `[B, B]`, not `[B, 1]`.

Views, copies, strides, and contiguity

A tensor usually owns or refers to a one-dimensional storage. Shape and strides describe how that storage is interpreted. Operations such as slicing and `transpose` can return views sharing storage. A view mutation may change the base tensor.

```
x = torch.arange(12).reshape(3, 4)
y = x.t()                # usually non-contiguous view
z = y.contiguous()       # contiguous copy if required
r = y.reshape(2, 6)      # view when possible, copy otherwise
```

- `view(...)` requires a stride-compatible layout.
- `reshape(...)` returns a view when possible and otherwise copies.
- `permute(...)` rearranges dimensions without immediately rearranging storage.
- `contiguous()` materializes data in the requested contiguous memory layout when necessary.

Interview angle: a non-contiguous tensor is valid. It only becomes a problem when an operation requires a specific layout or when the layout hurts performance.

Dtypes

Common dtypes include:

Dtype	Typical use
<code>torch.float32</code>	Default neural-network training
<code>torch.float64</code>	High-precision scientific work; usually slower and larger
<code>torch.float16</code>	GPU mixed precision; limited numeric range
<code>torch.bfloat16</code>	Mixed precision with float32-like exponent range
<code>torch.int64</code>	Class indices and many indexing operations
<code>torch.bool</code>	Masks

Class targets for `CrossEntropyLoss` are normally `torch.long` (`int64`), while model inputs are normally floating point.

Devices

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = torch.randn(32, 100, device=device)
model = model.to(device)
```

Moving the model does not move future batches automatically. Both model parameters and inputs must be placed correctly. Repeatedly moving small tensors inside a hot loop creates overhead.

4.2 Automatic differentiation (`autograd`)

Autograd computes vector-Jacobian products by reverse-mode automatic differentiation. During a differentiable forward pass, output tensors keep references to the operation that produced them. `backward()` traverses this graph in reverse topological order.

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = (x.square() + 2 * x).sum()
y.backward()
print(x.grad) # 2*x + 2 -> tensor([6., 8.])
```

Dynamic computation graphs

The graph is created as Python executes. Data-dependent branches and loops can therefore change the graph on every forward pass.

```
def piecewise(x):
    if x.mean() > 0:
        return x.square().sum()
    return x.abs().sum()
```

Dynamic does not mean autograd differentiates through arbitrary Python decisions. The selected branch's tensor operations are differentiated; the Boolean decision itself is not.

Leaf and non-leaf tensors

- A user-created tensor with `requires_grad=True` and no creating autograd operation is usually a **leaf**.
- Model parameters are leaf tensors.
- Intermediate results are usually **non-leaf** tensors.
- After backward, gradients are retained by default on leaves, not every intermediate. Use `retain_grad()` on a non-leaf only when debugging or explicitly needed.

Gradient accumulation

PyTorch adds new gradients into `.grad`; it does not overwrite them.

```
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

`set_to_none=True` often saves memory writes. It also makes parameters that received no gradient distinguishable because `param.grad` is `None`.

Gradient accumulation is useful when a desired effective batch does not fit in memory:

```
optimizer.zero_grad(set_to_none=True)
for step, (x, y) in enumerate(loader):
    loss = criterion(model(x), y) / accumulation_steps
    loss.backward()
    if (step + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

Dividing the loss preserves approximately the same gradient scale as the larger batch. The final partial group must also be stepped; production code should handle it explicitly.

Disabling graph construction

```
model.eval()
with torch.inference_mode():
    predictions = model(x)
```

- `torch.no_grad()` disables gradient recording inside its context.
- `torch.inference_mode()` additionally removes more autograd bookkeeping and is appropriate for pure inference when tensors created there will not later participate in tracked autograd work.
- `tensor.detach()` returns a tensor disconnected from the current graph; it can share storage.

`model.eval()` and `no_grad()` solve different problems. `eval()` changes module behavior such as dropout and batch normalization. `no_grad()` or `inference_mode()` changes autograd recording. Correct evaluation normally uses both.

In-place operations

Methods ending in `_`, such as `add_`, modify a tensor in place. They can reduce allocations but may destroy values autograd saved for backward or violate version checks. Avoid in-place mutation unless its correctness and benefit are clear.

Higher-order and selective gradients

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3
first, = torch.autograd.grad(y, x, create_graph=True)
second, = torch.autograd.grad(first, x)
print(first, second) # 12, 12
```

`torch.autograd.grad` returns gradients directly rather than accumulating into `.grad`. `create_graph=True` records the gradient computation for higher-order derivatives; it increases memory use.

4.3 `nn.Module`, parameters, buffers, and state

`nn.Module` is the base class for models and layers. It registers child modules, parameters, and persistent buffers so that device movement, serialization, mode changes, and parameter traversal work recursively.

```
from torch import nn

class MLP(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int, classes: int):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden_dim, classes),
        )

    def forward(self, x):
        return self.network(x)
```

Call `model(x)`, not `model.forward(x)`. `__call__` runs framework machinery such as hooks before and after delegating to `forward`.

Parameters

`nn.Parameter` is a tensor subclass that is automatically registered when assigned as a module attribute.

```
class Scale(nn.Module):
    def __init__(self, features):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(features))

    def forward(self, x):
        return x * self.weight
```

A plain tensor attribute is not a parameter and will not appear in `model.parameters()`.

Buffers

Buffers are non-parameter tensor state. They move with the model and can be saved in `state_dict`, but optimizers do not update them.

```
class Standardize(nn.Module):
    def __init__(self, mean, std):
        super().__init__()
        self.register_buffer("mean", torch.as_tensor(mean))
        self.register_buffer("std", torch.as_tensor(std))

    def forward(self, x):
        return (x - self.mean) / self.std.clamp_min(1e-6)
```

Batch normalization running means and variances are common buffers.

state_dict

A module's `state_dict` maps names to parameter and persistent-buffer tensors. An optimizer also has a `state_dict`, holding items such as momentum or Adam moments.

```
torch.save(model.state_dict(), "model.pt")

restored = MLP(input_dim=20, hidden_dim=64, classes=3)
state = torch.load("model.pt", map_location="cpu", weights_only=True)
restored.load_state_dict(state)
```

Saving a `state_dict` is usually more portable than serializing an entire Python model object. Architecture code and constructor arguments must still be versioned.

Train and evaluation modes

`model.train()` and `model.eval()` recursively set the `training` flag.

Layer	Training mode	Evaluation mode
Dropout	Randomly masks activations and rescales survivors	Identity
BatchNorm	Uses batch statistics and updates running statistics	Uses stored running statistics
Linear/ReLU	Same behavior	Same behavior

`eval()` does not freeze parameters. To freeze values, set `requires_grad_(False)` or exclude them from the optimizer.

4.4 Layers, activations, and output conventions

Frequently used layers:

Layer	Expected role	Typical shape convention
<code>nn.Linear</code>	Dense affine transformation	$[..., \text{in_features}] \rightarrow [..., \text{out_features}]$
<code>nn.Conv2d</code>	Spatial feature extraction	$[N, C_{\text{in}}, H, W] \rightarrow [N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}}]$
<code>nn.Embedding</code>	Integer ID to learned vector	$[...] \rightarrow [..., \text{embedding_dim}]$
<code>nn.LSTM</code> / <code>nn.GRU</code>	Recurrent sequence modeling	Often $[N, T, D]$ with <code>batch_first=True</code>
<code>nn.MultiheadAttention</code>	Attention over sequences	Often $[N, T, D]$ with <code>batch_first=True</code>
<code>nn.LayerNorm</code>	Normalize each sample over final dimensions	Shape preserved
<code>nn.BatchNorm*</code>	Normalize channels using batch statistics	Shape preserved
<code>nn.Dropout</code>	Stochastic regularization	Shape preserved

Typical activation choices:

- **ReLU:** simple and common in MLPs/CNNs.
- **GELU:** common in Transformers.
- **SilU/Swish:** common in modern vision and generative models.
- **Sigmoid:** maps a scalar logit to a binary probability; often omitted from the model during training when using `BCEWithLogitsLoss`.
- **Softmax:** maps class logits to a distribution; often omitted during training when using `CrossEntropyLoss`.

Logits versus probabilities

A **logit** is an unnormalized score. Prefer numerically stable combined losses:

Task	Model output	Target	Loss	Inference conversion
Regression	<code>[N]</code> or <code>[N, d]</code> value	float, same shape	MSE/MAE/Huber	none
Binary class	<code>[N]</code> logits	float 0/1, same shape	<code>BCEWithLogitsLoss</code>	<code>sigmoid(logits)</code>
Multiclass	<code>[N, C]</code> logits	long class IDs <code>[N]</code>	<code>CrossEntropyLoss</code>	<code>softmax(logits, dim=1)</code>
Multilabel	<code>[N, C]</code> logits	float multi-hot <code>[N, C]</code>	<code>BCEWithLogitsLoss</code>	independent sigmoid

Do not apply softmax before `CrossEntropyLoss`; it internally combines log-softmax and negative log-likelihood. Do not apply sigmoid before `BCEWithLogitsLoss`; the combined implementation is more stable.

4.5 Datasets and data loading

Map-style datasets

A map-style `Dataset` implements `__len__` and `__getitem__`.

```
from torch.utils.data import Dataset

class TabularDataset(Dataset):
    def __init__(self, features, targets):
        self.features = torch.as_tensor(features, dtype=torch.float32)
        self.targets = torch.as_tensor(targets, dtype=torch.long)

    def __len__(self):
        return len(self.targets)

    def __getitem__(self, index):
        return self.features[index], self.targets[index]
```

Iterable datasets

`IterableDataset` yields samples sequentially and suits streams, very large files, or sources without random access. With multiple workers, each worker must be sharded to avoid duplicate samples.

DataLoader

```
from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=128,
```

```

shuffle=True,
num_workers=4,
pin_memory=True,
persistent_workers=True,
)

```

Key options:

Option	Meaning	Practical note
<code>batch_size</code>	Samples per batch	Larger improves throughput until memory or generalization becomes limiting
<code>shuffle</code>	Randomizes sample order	Usually true only for training
<code>sampler</code>	Controls index sequence	Cannot generally be combined with <code>shuffle=True</code>
<code>num_workers</code>	Loader subprocess count	Tune; more is not always faster
<code>collate_fn</code>	Combines samples into a batch	Needed for padding or variable structures
<code>pin_memory</code>	Uses page-locked host memory	Can speed CPU-to-CUDA copies
<code>drop_last</code>	Drops incomplete final batch	Sometimes useful for fixed batch behavior
<code>persistent_workers</code>	Reuses workers across epochs	Avoids repeated startup when workers > 0

For variable-length text, a collate function can pad within each batch:

```

from torch.nn.utils.rnn import pad_sequence

def collate_tokens(samples):
    sequences, labels = zip(*samples)
    lengths = torch.tensor([len(seq) for seq in sequences])
    padded = pad_sequence(sequences, batch_first=True, padding_value=0)
    return padded, lengths, torch.tensor(labels)

```

Data transforms derived from the dataset, such as mean/std scaling or vocabulary fitting, must be fit on the training split only.

4.6 Loss functions

A loss is the differentiable training objective. A metric is a human-facing measure and need not be differentiable.

Loss	Use	Common mistake
<code>MSELoss</code>	Regression, Gaussian-error assumption	Sensitive to outliers
<code>L1Loss</code>	Robust regression	Gradient is not smooth at zero
<code>SmoothL1Loss</code> / Huber	Balanced regression robustness	Transition parameter needs thought
<code>CrossEntropyLoss</code>	Single-label multiclass	Applying softmax first or one-hot target unexpectedly
<code>BCEWithLogitsLoss</code>	Binary/multilabel	Passing integer targets or already-sigmoided values
<code>NLLLoss</code>	Inputs are log-probabilities	Forgetting <code>log_softmax</code>
<code>KLDivLoss</code>	Distribution matching	Input convention/reduction misunderstood
Contrastive/triplet losses	Representation learning	Poor negative sampling

Class weights can alter the optimization objective for imbalance, but they do not repair a biased split or missing population coverage.

4.7 Optimizers and schedulers

An optimizer reads parameter gradients and updates parameter values.

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,
    weight_decay=1e-2,
)
```

Common optimizers:

Optimizer	Strength	Typical use/interview point
SGD	Simple, memory efficient	Often strong generalization; momentum accelerates consistent directions
Adam	Adaptive per-parameter scaling	Fast default for many problems
AdamW	Decoupled weight decay	Common default for Transformers and modern deep learning

Optimizer	Strength	Typical use/interview point
RMSprop	Adaptive squared-gradient average	Historically common in recurrent/RL settings
SparseAdam	Sparse gradients	Useful for certain sparse embeddings

Parameter groups support different hyperparameters:

```
optimizer = torch.optim.AdamW([
    {"params": model.backbone.parameters(), "lr": 1e-5},
    {"params": model.classifier.parameters(), "lr": 1e-3},
], weight_decay=1e-2)
```

A scheduler changes learning rates over training. Examples include step decay, cosine decay, one-cycle policies, warm-up, and plateau-based reduction. Scheduler call timing matters: most schedulers step after `optimizer.step()`, while `ReduceLROnPlateau` steps after validation with a metric.

4.8 The training loop

The essential order is:

```
model.train()
for inputs, targets in train_loader:
    optimizer.zero_grad(set_to_none=True)
    logits = model(inputs)
    loss = criterion(logits, targets)
    loss.backward()
    optimizer.step()
```

Why the order matters:

1. Old gradients are cleared.
2. Forward computation creates predictions and a graph.
3. Loss converts prediction quality into a scalar objective.
4. Backward computes and accumulates gradients.
5. The optimizer updates parameters.

Evaluation should not mutate model parameters and should aggregate statistics correctly across all examples.

```
model.eval()
total_loss = 0.0
correct = 0

with torch.inference_mode():
    for inputs, targets in validation_loader:
```

```
logits = model(inputs)
loss = criterion(logits, targets)
total_loss += loss.item() * targets.size(0)
correct += (logits.argmax(dim=1) == targets).sum().item()
```

Multiplying a mean batch loss by batch size before summing prevents the last smaller batch from receiving equal weight to a full batch.

4.9 Initialization and normalization

Initialization controls early activation and gradient scales.

- Xavier/Glorot initialization is designed around preserving variance for roughly symmetric activations.
- Kaiming/He initialization is designed for ReLU-like activations.
- PyTorch layers already have sensible defaults, so custom initialization should have a reason.

```
def initialize(module):
    if isinstance(module, nn.Linear):
        nn.init.kaiming_normal_(module.weight, nonlinearity="relu")
        if module.bias is not None:
            nn.init.zeros_(module.bias)

model.apply(initialize)
```

Normalization comparison:

Method	Normalizes over	Dependency on other samples	Common use
BatchNorm	Batch and spatial axes per channel	Yes in training	CNNs with sufficiently large batches
LayerNorm	Feature axes within each sample	No	Transformers, sequence models
GroupNorm	Channel groups within each sample	No	Vision with small batches
InstanceNorm	Each channel of each sample	No	Style transfer and some generative vision

4.10 Mixed precision and numerical stability

Automatic mixed precision (AMP) runs eligible operations in lower precision while keeping sensitive work in safer precision. It can reduce activation memory and improve accelerator throughput.

```

use_amp = device.type == "cuda"
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

optimizer.zero_grad(set_to_none=True)
with torch.amp.autocast(device_type=device.type, enabled=use_amp):
    logits = model(inputs)
    loss = criterion(logits, targets)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

```

Float16 gradients can underflow. Gradient scaling multiplies the loss before backward, then unscales gradients before the optimizer update. With clipping, unscale first:

```

scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
scaler.step(optimizer)
scaler.update()

```

Numerical-stability habits:

- Use combined logit losses.
- Prefer `logsumexp` over manually computing `log(exp(x).sum())`.
- Clamp only when it reflects the mathematical domain; blind clamping can hide bugs.
- Monitor non-finite losses and gradients.
- Use `torch.autograd.detect_anomaly()` briefly for debugging, not normal training.

4.11 Reproducibility

```

import random
import numpy as np
import torch

def seed_everything(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

```

Seeds make random streams repeatable, but exact reproducibility can still be affected by hardware, library versions, parallel reductions, data-loader workers, nondeterministic kernels, and distributed execution.

When exact reproducibility is required:

```
torch.use_deterministic_algorithms(True)
```

Deterministic algorithms can be slower or unavailable for some operations. For serious experiments, save code version, dependency versions, seeds, split IDs, preprocessing state, hyperparameters, and checkpoint—not merely the model weights.

4.12 Saving and resuming training

A resumable checkpoint typically contains:

```
checkpoint = {  
    "epoch": epoch,  
    "model": model.state_dict(),  
    "optimizer": optimizer.state_dict(),  
    "scheduler": scheduler.state_dict(),  
    "best_validation_loss": best_validation_loss,  
    "config": config,  
}  
torch.save(checkpoint, "checkpoint.pt")
```

On load, reconstruct the model and optimizer first, then load their states. Move optimizer-state tensors to the required device if the loading workflow does not do so. A true exact resume may also require scaler state, sampler state, and random-number-generator states.

Security interview point: checkpoint files are inputs. Use trusted files and safer weight-only loading paths where applicable; unrestricted Python object deserialization can execute code.

4.13 Compilation, export, and inference

Eager execution is simple and debuggable. Compilation can capture and optimize regions of tensor work:

```
compiled_model = torch.compile(model)
```

Compilation may fuse operations and reduce Python overhead, but first-call compilation costs, graph breaks, dynamic shapes, unsupported operations, and backend differences must be measured. Benchmark warmed-up end-to-end workloads, not one isolated call.

An inference pipeline needs more than a forward pass:

1. Load version-matched architecture and weights.
2. Reproduce preprocessing exactly.
3. Set evaluation mode.

4. Disable autograd.
5. Batch requests when latency constraints allow.
6. Convert logits to the required output representation.
7. Apply thresholding/calibration/postprocessing.
8. Monitor input drift, latency, failures, and output quality.

Export is distinct from saving a checkpoint. A checkpoint preserves PyTorch state for PyTorch code; an exported graph targets a constrained runtime or deployment interface.

4.14 Multi-GPU and distributed training

DistributedDataParallel (DDP) runs one process per device, keeps a model replica in each process, and synchronizes gradients—usually with all-reduce—during backward.

Conceptually:

```
process/device 0: local batch → local gradients ↗
process/device 1: local batch → local gradients ↗ → all-reduce/average → same
update
process/device k: local batch → local gradients ↘
```

Key points:

- Use a distributed sampler so each process receives a different data shard.
- Call `sampler.set_epoch(epoch)` so shuffling changes consistently each epoch.
- Only one rank should normally write a shared checkpoint or log a global result.
- Metrics must be reduced across processes; local accuracy is not global accuracy.
- Effective global batch size is approximately `batch_per_device × number_of_processes × accumulation_steps`.

Minimal structure:

```
import os
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, DistributedSampler

dist.init_process_group(backend="nccl")
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)

model = MyModel().to(local_rank)
model = DDP(model, device_ids=[local_rank])
sampler = DistributedSampler(train_dataset, shuffle=True)
loader = DataLoader(train_dataset, sampler=sampler, shuffle=False)

for epoch in range(num_epochs):
    sampler.set_epoch(epoch)
```

```
train_one_epoch(model, loader)

dist.destroy_process_group()
```

DDP replicates the full model. Fully sharded approaches shard parameters, gradients, and optimizer state to train models too large for one device. Tensor parallelism splits individual tensor operations; pipeline parallelism splits layer stages; data parallelism splits samples.

4.15 Debugging and observability

A practical debugging order:

1. Overfit a tiny batch. Failure suggests an implementation, capacity, or optimization bug.
2. Print/assert input, target, output, and loss shapes.
3. Check dtypes, devices, label ranges, and non-finite values.
4. Verify train/eval mode and gradient context.
5. Inspect whether every expected parameter has a gradient.
6. Inspect gradient and activation magnitudes.
7. Confirm the optimizer contains the intended parameters.
8. Compare against a simple baseline.

Useful checks:

```
assert logits.shape == (targets.size(0), num_classes)
assert targets.dtype == torch.long
assert torch.isfinite(loss)

for name, parameter in model.named_parameters():
    if parameter.requires_grad and parameter.grad is None:
        print("No gradient:", name)
```

Hooks can inspect internal activations or gradients, but they should be removed after debugging because they can retain tensors, add overhead, or make behavior harder to reason about.

5. Algorithm / Working Process

PyTorch training is a coordinated data, forward, differentiation, and update process.

Step 1: Define the task contract

Decide:

- Input representation and exact tensor shape.
- Target representation.
- Model output: raw value, logit, class logits, embedding, sequence, mask, etc.
- Loss and metrics.

- Validation protocol and success threshold.

Example multiclass contract:

```
Input: float tensor X with shape [batch, features]
Target: int64 tensor y with shape [batch], values in [0, classes - 1]
Output: float logits with shape [batch, classes]
Loss: cross entropy
Metric: accuracy plus per-class precision/recall when imbalance matters
```

Step 2: Prepare and split data

Split using the unit that will be independent at deployment. For medical images, splitting images randomly can leak the same patient across sets; split by patient. For time series, use chronological validation rather than a random split. Fit preprocessing only on training data.

Step 3: Build datasets and loaders

The dataset returns one sample. The loader batches, shuffles, parallelizes reads, and optionally pins memory. Training is usually shuffled; validation and test are usually deterministic.

Step 4: Define an `nn.Module`

Create layers in `__init__`; define data flow in `forward`. Parameters are registered through assigned modules or `nn.Parameter` objects.

Step 5: Choose loss and optimizer

The loss must match the output/target contract. The optimizer receives the exact parameters to update.

Step 6: Forward pass

For a batch (X), the model computes predictions:

```
Z = f_theta(X)
```

Here, `theta` denotes all trainable parameters. Autograd records operations involving tensors that require gradients.

Step 7: Loss computation

```
L = loss_fn(Z, y)
```

The training loss should usually reduce to a scalar. For custom weighting or masking, compute unreduced per-example/token losses, apply the mask and weights deliberately, then reduce with the correct denominator.

Step 8: Backward pass

`L.backward()` starts with $(dL/dL = 1)$ for a scalar loss and applies the chain rule backward. Parameter `.grad` fields receive accumulated gradients.

Step 9: Parameter update

The optimizer applies its rule, such as SGD or AdamW. This update is normally performed under no-grad semantics internally so it does not become part of the next computation graph.

Step 10: Validation

Set `eval()` and use `inference_mode()`. Compute loss and task metrics over the full validation set. Do not tune on the test set.

Step 11: Checkpoint and stop

Save the best model according to a validation criterion. Early stopping is a training-control rule, not a substitute for a separate test set.

Step 12: Inference

Use identical feature schema and preprocessing, produce outputs, convert logits when needed, and apply thresholds or postprocessing chosen on validation data.

Training and inference comparison

Stage	model mode	Autograd	Random augmentation	Parameter update
Training	<code>train()</code>	Enabled	Often enabled	Yes
Validation	<code>eval()</code>	Disabled	Disabled	No
Test	<code>eval()</code>	Disabled	Disabled	No
Inference	<code>eval()</code>	Disabled	Disabled	No

6. Mathematical Foundation

6.1 Tensors and affine layers

For a batch $(X \in \mathbb{R}^{B \times d_{in}})$, a linear layer with weights $(W \in \mathbb{R}^{d_{out} \times d_{in}})$ and bias $(b \in \mathbb{R}^{d_{out}})$ computes:

$$Z = XW^T + b$$

The bias broadcasts across the (B) examples. PyTorch's `nn.Linear(d_in, d_out)` stores weights with shape `[d_out, d_in]`.

Parameter count:

$$d_{out}d_{in} + d_{out}$$

6.2 Activations

ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Softmax for class (c):

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

For numerical stability, implementations shift logits by their maximum before exponentiation. The probability is unchanged because the same factor cancels from numerator and denominator.

6.3 Loss functions

Mean squared error

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

MSE strongly penalizes large residuals. Minimizing MSE estimates the conditional mean under common assumptions.

Binary cross entropy with logits

For binary target ($y \in \{0, 1\}$) and logit (z):

$$\mathcal{L}_{BCE} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$$

`BCEWithLogitsLoss` evaluates an algebraically stable form rather than explicitly materializing potentially saturated probabilities.

Multiclass cross entropy

For logits ($z_{\{i,c\}}$) and correct class (y_i):

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \log \left(\frac{e^{z_{i,y_i}}}{\sum_{c=1}^C e^{z_{i,c}}} \right)$$

The gradient with respect to a sample's class logit is:

$$\frac{\partial \mathcal{L}}{\partial z_c} = p_c - 1[c = y]$$

This elegant result explains the update: predicted probability is reduced for wrong classes and increased toward one for the correct class.

Label smoothing

Instead of a one-hot target, label smoothing allocates most mass to the correct class and a small amount elsewhere. It can reduce overconfidence, but excessive smoothing can hurt calibration or hard-class learning.

6.4 Backpropagation and the chain rule

For a composition ($y=f(g(x))$):

$$\frac{dy}{dx} = \frac{dy}{dg} \frac{dg}{dx}$$

For a two-layer network:

$$h = \phi(XW_1^T + b_1), \quad z = hW_2^T + b_2$$

autograd starts from the loss derivative at (z), propagates through the second affine layer, through activation derivative (ϕ'), and through the first layer. Reverse mode is efficient when many parameters lead to one scalar loss.

PyTorch generally computes vector-Jacobian products rather than constructing a full Jacobian. For non-scalar output y , `y.backward(v)` supplies the vector (v) in ($v^T J$).

6.5 Gradient descent, momentum, Adam, and AdamW

Basic SGD:

$$\theta_{t+1} = \theta_t - \eta g_t$$

where ($g_t = \nabla_{\theta} \mathcal{L}_t$) and (η) is the learning rate.

Momentum maintains a velocity:

$$v_t = \mu v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t$$

Adam keeps exponential moving averages of first and second gradient moments:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias-corrected moments:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

AdamW decouples weight decay from the adaptive gradient update, conceptually adding a shrinkage step:

$$\theta \leftarrow (1 - \eta\lambda)\theta - \text{adaptive-update}$$

This is not identical to adding L2 regularization to Adam's loss because Adam rescales gradients coordinate-wise.

6.6 Batch normalization

For mini-batch values ($x_1, \dots, x_m$):

$$\mu_B = \frac{1}{m} \sum_i x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_i (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

At evaluation, BatchNorm normally uses running statistics accumulated during training. This is why missing `eval()` can make inference depend on request batch composition.

6.7 Convolution

For a 2D input, a convolution/cross-correlation computes a weighted local sum. Output size per spatial dimension is:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - D(K - 1) - 1}{S} + 1 \right\rfloor$$

where (K) is kernel size, (S) stride, (P) padding, and (D) dilation.

Conv2d parameter count:

$$C_{out} \left(\frac{C_{in}}{G} K_h K_w + 1_{bias} \right)$$

where (G) is the number of groups.

6.8 Attention

Scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

The mask (M) may block padding positions or future tokens. The ($\sqrt{d_k}$) scaling keeps dot products from becoming too large as dimension grows.

For sequence length (T), dense self-attention has roughly ($O(T^2 d)$) compute in its attention component and ($O(T^2)$) attention-score memory, motivating memory-efficient kernels and sparse/linear variants.

6.9 Regularization

L2-style penalty:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \|\theta\|_2^2$$

Dropout during training samples a mask ($m_i \sim \text{Bernoulli}(1-p)$) and uses inverted scaling:

$$y_i = \frac{m_i x_i}{1 - p}$$

This preserves the expected activation, so no rescaling is required at evaluation.

6.10 Gradient clipping

Global norm clipping rescales gradients when their combined norm exceeds threshold (c):

$$g \leftarrow g \cdot \min \left(1, \frac{c}{\|g\| + \epsilon} \right)$$

It is useful for exploding gradients, especially in recurrent or unstable deep networks, but should not be used to hide an unsuitable learning rate or broken loss.

7. Practical Implementation

The following is a complete, self-contained multiclass classification example. It creates a synthetic dataset so it can run without downloading data, performs a stratified train/validation/test split, standardizes using training statistics only, trains an MLP, checkpoints the best validation model, reloads it, and evaluates the untouched test set.

```
from __future__ import annotations

import copy
import random
from dataclasses import dataclass

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

@dataclass(frozen=True)
class Config:
    seed: int = 42
    samples: int = 6000
    features: int = 20
    classes: int = 3
    hidden: int = 64
    batch_size: int = 128
    epochs: int = 30
    learning_rate: float = 3e-3
    weight_decay: float = 1e-4
    patience: int = 5

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)

def make_dataset(config: Config) -> tuple[torch.Tensor, torch.Tensor]:
    """Create learnable nonlinear multiclass data."""
    generator = torch.Generator().manual_seed(config.seed)
    x = torch.randn(config.samples, config.features, generator=generator)

    true_w = torch.randn(config.features, config.classes, generator=generator)
    linear_scores = x @ true_w
    nonlinear_scores = torch.stack(
        (
            x[:, 0] * x[:, 1],
            x[:, 2].square() - x[:, 3],
            torch.sin(x[:, 4]) + x[:, 5] * x[:, 6],
        ),
        dim=1,
    )
    logits = linear_scores + 1.5 * nonlinear_scores
    logits += 0.5 * torch.randn(logits.shape, generator=generator)
    y = logits.argmax(dim=1)
    return x, y

def stratified_indices(
    y: torch.Tensor,
    train_fraction: float,
    validation_fraction: float,
    seed: int,
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """Split every class separately to preserve class proportions."""
    generator = torch.Generator().manual_seed(seed)
    train_parts, validation_parts, test_parts = [], [], []

    for class_id in y.unique(sorted=True):
        indices = torch.where(y == class_id)[0]
        indices = indices[torch.randperm(len(indices), generator=generator)]
        train_end = int(train_fraction * len(indices))
        validation_end = train_end + int(validation_fraction * len(indices))
        train_parts.append(indices[:train_end])
        validation_parts.append(indices[train_end:validation_end])
        test_parts.append(indices[validation_end:])

    def shuffled(parts: list[torch.Tensor]) -> torch.Tensor:
        joined = torch.cat(parts)
        return joined[torch.randperm(len(joined), generator=generator)]

    return shuffled(train_parts), shuffled(validation_parts), shuffled(test_parts)

class Classifier(nn.Module):
    def __init__(self, features: int, hidden: int, classes: int) -> None:
        super().__init__()

```

```

        self.network = nn.Sequential(
            nn.Linear(features, hidden),
            nn.ReLU(),
            nn.Dropout(p=0.15),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.network(x)

def make_loader(
    x: torch.Tensor,
    y: torch.Tensor,
    indices: torch.Tensor,
    batch_size: int,
    shuffle: bool,
    use_cuda: bool,
) -> DataLoader:
    dataset = TensorDataset(x[indices], y[indices])
    return DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        num_workers=0, # portable default; benchmark >0 for real input pipelines
        pin_memory=use_cuda,
    )

def run_epoch(
    model: nn.Module,
    loader: DataLoader,
    criterion: nn.Module,
    device: torch.device,
    optimizer: torch.optim.Optimizer | None = None,
) -> tuple[float, float]:
    is_training = optimizer is not None
    model.train(is_training)

    total_loss = 0.0
    total_correct = 0
    total_examples = 0

    context = torch.enable_grad() if is_training else torch.inference_mode()
    with context:
        for features, targets in loader:
            features = features.to(device, non_blocking=True)
            targets = targets.to(device, non_blocking=True)

            if is_training:
                optimizer.zero_grad(set_to_none=True)

```

```

        logits = model(features)
        loss = criterion(logits, targets)

        if is_training:
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
            optimizer.step()

        batch_size = targets.size(0)
        total_loss += loss.item() * batch_size
        total_correct += (logits.argmax(dim=1) == targets).sum().item()
        total_examples += batch_size

    return total_loss / total_examples, total_correct / total_examples

def confusion_matrix(
    model: nn.Module,
    loader: DataLoader,
    classes: int,
    device: torch.device,
) -> torch.Tensor:
    matrix = torch.zeros(classes, classes, dtype=torch.int64)
    model.eval()
    with torch.inference_mode():
        for features, targets in loader:
            features = features.to(device)
            predictions = model(features).argmax(dim=1).cpu()
            for target, prediction in zip(targets, predictions):
                matrix[target, prediction] += 1
    return matrix

def main() -> None:
    config = Config()
    seed_everything(config.seed)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    x, y = make_dataset(config)
    train_idx, validation_idx, test_idx = stratified_indices(
        y,
        train_fraction=0.70,
        validation_fraction=0.15,
        seed=config.seed,
    )

    # Fit preprocessing on training data only.
    train_mean = x[train_idx].mean(dim=0, keepdim=True)
    train_std = x[train_idx].std(dim=0, keepdim=True).clamp_min(1e-6)
    x = (x - train_mean) / train_std

    use_cuda = device.type == "cuda"
    train_loader = make_loader(
        x, y, train_idx, config.batch_size, shuffle=True, use_cuda=use_cuda

```

```

)
validation_loader = make_loader(
    x, y, validation_idx, config.batch_size, shuffle=False, use_cuda=use_cuda
)
test_loader = make_loader(
    x, y, test_idx, config.batch_size, shuffle=False, use_cuda=use_cuda
)

model = Classifier(config.features, config.hidden, config.classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=config.learning_rate,
    weight_decay=config.weight_decay,
)

best_state = None
best_validation_loss = float("inf")
epochs_without_improvement = 0

for epoch in range(1, config.epochs + 1):
    train_loss, train_accuracy = run_epoch(
        model, train_loader, criterion, device, optimizer
    )
    validation_loss, validation_accuracy = run_epoch(
        model, validation_loader, criterion, device
    )

    print(
        f"epoch={epoch:02d} "
        f"train_loss={train_loss:.4f} train_acc={train_accuracy:.3f} "
        f"val_loss={validation_loss:.4f} val_acc={validation_accuracy:.3f}"
    )

    if validation_loss < best_validation_loss:
        best_validation_loss = validation_loss
        best_state = copy.deepcopy(model.state_dict())
        epochs_without_improvement = 0
    else:
        epochs_without_improvement += 1
        if epochs_without_improvement >= config.patience:
            print("Early stopping")
            break

if best_state is None:
    raise RuntimeError("Training produced no checkpoint")

model.load_state_dict(best_state)
test_loss, test_accuracy = run_epoch(model, test_loader, criterion, device)
matrix = confusion_matrix(model, test_loader, config.classes, device)

print(f"test_loss={test_loss:.4f} test_acc={test_accuracy:.3f}")
print("confusion matrix (rows=true, columns=predicted):")
print(matrix)

```

```

torch.save(
    {
        "model": model.state_dict(),
        "feature_mean": train_mean,
        "feature_std": train_std,
        "config": config.__dict__,
    },
    "pytorch_classifier.pt",
)

if __name__ == "__main__":
    main()

```

Focused implementation: CUDA mixed precision

Replace the training part of a batch with:

```

use_amp = device.type == "cuda"
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

optimizer.zero_grad(set_to_none=True)
with torch.amp.autocast(device_type=device.type, enabled=use_amp):
    logits = model(features)
    loss = criterion(logits, targets)

scaler.scale(loss).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
scaler.step(optimizer)
scaler.update()

```

Focused implementation: transfer learning with torchvision

```

import torch
from torch import nn
from torchvision.models import ResNet18_Weights, resnet18

weights = ResNet18_Weights.DEFAULT
preprocess = weights.transforms()
model = resnet18(weights=weights)

for parameter in model.parameters():
    parameter.requires_grad = False

model.fc = nn.Linear(model.fc.in_features, 5)

```

```
model = model.to(device)
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
```

The training images must receive preprocessing compatible with the pretrained weights. For fine-tuning later, unfreeze some or all backbone layers, lower their learning rate, and rebuild or extend optimizer parameter groups.

Focused implementation: safe inference function

```
def predict_proba(
    model: nn.Module,
    features: torch.Tensor,
    device: torch.device,
) -> torch.Tensor:
    model.eval()
    with torch.inference_mode():
        logits = model(features.to(device))
        return logits.softmax(dim=-1).cpu()
```

8. Code Explanation

Configuration

`Config` centralizes experiment values so the run is easy to reproduce. These values are hyperparameters, not learned parameters. A frozen dataclass prevents accidental mutation during the experiment.

Data generation

`make_dataset` creates both linear and nonlinear class structure. A purely linear dataset would make the second hidden layer unnecessary and would not demonstrate an MLP's additional capacity.

Stratified splitting

`stratified_indices` splits each class independently, keeping class proportions similar. Test indices are isolated before training. The same seed yields the same split.

For grouped, temporal, or user-level data, this function would be insufficient; the split must respect the real independence boundary.

Preprocessing

Mean and standard deviation are computed only from `x[train_idx]`. Applying those fixed values to validation and test is correct. Computing statistics from all data would leak information from evaluation sets.

`clamp_min(1e-6)` protects against constant or near-constant features causing division by zero.

Data loaders

Training uses `shuffle=True`; validation and test do not. `pin_memory` is enabled only for CUDA, and `.to(..., non_blocking=True)` can overlap compatible host-to-device copies with computation when the pipeline supports it.

`num_workers=0` is the most portable example default. Real projects should benchmark loader worker counts because optimal values depend on storage, transforms, CPU count, batch size, and operating system.

Model

The model emits three raw logits. There is no final softmax because `CrossEntropyLoss` expects logits. Dropout is active during `train()` and disabled during `eval()`.

run_epoch

Passing an optimizer selects training behavior. Without an optimizer, the function switches to evaluation and inference mode. This keeps the aggregation logic identical while preserving the key semantic differences.

The loss is multiplied by batch size because the criterion returns a batch mean. Summing raw batch means would bias the epoch result when batch sizes differ.

Backpropagation

`zero_grad(set_to_none=True)` clears old gradients. `loss.backward()` fills gradients. Gradient clipping protects the example against unusually large norms. `optimizer.step()` changes parameters.

Early stopping

The best validation state is deep-copied. A plain assignment such as `best_state = model.state_dict()` can leave references tied to tensors that continue changing. Test evaluation occurs only after model selection is finished.

Confusion matrix

Rows represent actual labels and columns predicted labels. Diagonal values are correct predictions. Off-diagonal cells identify specific class confusions that overall accuracy hides.

The Python loop is intentionally readable. For very large evaluation sets, encode pairs as `target * classes + prediction` and use `torch.bincount` for a vectorized implementation.

Checkpoint

The saved artifact contains model weights and training-derived preprocessing state. In a real system, also record class-to-index mapping, feature schema/version, library versions, model code version, and decision thresholds.

9. Training / Evaluation

9.1 Dataset preparation

1. Define a sample and label unambiguously.

2. Validate corrupted, missing, duplicated, and mislabeled records.
3. Split at the correct unit—patient, customer, document, session, time window, or source.
4. Fit preprocessing and tokenizers on the training portion only when they learn from data.
5. Use training-only stochastic augmentation.
6. Store the exact preprocessing and label mapping with the model.

9.2 Train, validation, and test roles

Split	Purpose	May influence model choice?
Train	Estimate parameters	Yes
Validation	Select hyperparameters, epoch, threshold	Yes
Test	Final unbiased estimate	No

Repeatedly reporting test performance while changing the model turns the test set into a validation set.

9.3 Metrics

Choose metrics based on the cost of errors and class distribution.

Classification

- Accuracy: useful for balanced classes with similar error costs.
- Precision: among predicted positives, how many are positive.
- Recall/sensitivity: among actual positives, how many are found.
- Specificity: among actual negatives, how many are rejected.
- F1: harmonic mean of precision and recall.
- ROC-AUC: ranking across thresholds; can look optimistic for extreme imbalance.
- PR-AUC: often more informative for rare positives.
- Log loss: evaluates probabilistic confidence.
- Calibration error/Brier score: assesses probability reliability.

For multiclass metrics, specify micro, macro, or weighted averaging. Macro gives each class equal weight; micro aggregates decisions; weighted averages by support.

Regression

- MAE for interpretable absolute error and robustness.
- RMSE when large errors deserve more penalty.
- (R^2) relative to a mean baseline.
- Domain metrics such as MAPE only when their assumptions are safe; MAPE is unstable near zero.

Ranking, generation, and dense prediction

- Recommenders/search: Recall@K, NDCG@K, MRR.
- Segmentation: IoU/Jaccard, Dice.
- Detection: mAP across IoU thresholds.
- Language models: negative log-likelihood/perplexity plus task and human evaluation.

- Generation: automatic metrics should be paired with factuality, safety, diversity, and human judgments when relevant.

9.4 Overfitting and underfitting

Pattern	Likely diagnosis	Responses
High train loss and high validation loss	Underfitting/optimization failure	More capacity, better features, train longer, tune LR, debug gradients
Low train loss but high validation loss	Overfitting or distribution mismatch	More data/augmentation, regularization, smaller model, early stopping, inspect split
Both improve then validation worsens	Overtraining	Checkpoint best validation epoch, regularize
Loss becomes NaN	Numerical/data/optimization issue	Inspect data, lower LR, stable loss, AMP/scaler, gradient norms
Accuracy flat at chance	Contract or learning bug	Check labels, output size, parameter registration, optimizer, tiny-batch overfit

9.5 Important hyperparameters

Hyperparameter	Effect	Tuning guidance
Learning rate	Update magnitude	Often the most important; search log scale
Batch size	Noise, throughput, memory	Scale only with validation evidence; may require LR change
Weight decay	Parameter shrinkage	Tune separately; often exclude bias/norm parameters in large models
Depth/width	Model capacity and cost	Increase only if optimization/data support it
Dropout	Stochastic regularization	Too high causes underfitting
Scheduler/warm-up	Learning-rate trajectory	Warm-up helps large batches/Transformers
Epochs/patience	Training duration	Choose using validation, not test
Gradient clip	Maximum gradient norm/value	Monitor how often clipping activates

9.6 Improving performance methodically

1. Build a simple baseline and verify the metric implementation.
2. Make a tiny subset overfit.
3. Establish a reproducible full-data run.
4. Inspect errors, data quality, and class/subgroup metrics.
5. Tune learning rate before broad architecture searches.

6. Improve data or labels when error analysis points there.
7. Use pretrained models when domain transfer is plausible.
8. Add AMP, compilation, or distributed training only after profiling.
9. Run ablations so improvements have evidence.
10. Report multiple seeds or confidence intervals when variance matters.

9.7 Gradient accumulation and effective batch size

If each device uses batch size (B), there are (D) data-parallel processes, and gradients accumulate for (A) micro-batches, the approximate effective batch is:

$$B_{effective} = BDA$$

This equivalence is imperfect when BatchNorm, dropout randomness, data order, clipping timing, or optimizer/scheduler updates differ.

9.8 Threshold selection and calibration

For binary prediction, 0.5 is not universally optimal. Select a threshold on validation data using domain costs, required precision/recall, or a utility function. Never choose it on the test set.

Softmax or sigmoid values need not be calibrated probabilities. Temperature scaling or other calibration methods can be fit on held-out validation data.

10. Complexity and Cost

10.1 Main memory consumers during training

Training memory includes:

1. Parameters.
2. Gradients.
3. Optimizer state.
4. Saved activations for backward.
5. Temporary workspaces and framework overhead.
6. Input batches and prefetched data.

For (P) parameters, float32 weights alone require approximately (4P) bytes. Standard Adam commonly adds two moment tensors, and training also stores gradients; mixed-precision implementations may retain master weights. Therefore total training state is several times model weight size even before activations.

10.2 Layer costs

Operation	Approximate compute	Key memory driver
Linear $[B, d_{in}] \rightarrow [B, d_{out}]$	$(O(Bd_{in}d_{out}))$	Activations and weights
Conv2d	$(O(BH_{out}W_{out}C_{out}(C_{in}/G)K_hK_w))$	Feature maps and kernels

Operation	Approximate compute	Key memory driver
Dense self-attention	$(O(BT^2d + BTd^2))$	Attention scores $(O(BT^2))$ plus activations
Embedding lookup	$(O(BT))$ lookups	Embedding table $(O(Vd))$
Elementwise activation	$(O(n))$	Tensor size

Backward often costs roughly the same order as, and commonly more than, forward because it computes input and parameter gradients and needs saved activations.

10.3 CPU versus GPU

CPU is reasonable for small models, small batches, preprocessing, debugging, and latency workloads that do not saturate a GPU. GPUs excel at large parallel tensor operations. Tiny batches or frequent CPU-GPU synchronization can leave a GPU underutilized.

10.4 Throughput versus latency

- Larger batches usually increase throughput but may increase per-request latency.
- Compilation can reduce overhead but adds warm-up/compile cost.
- Data-loader workers help only if input loading is a bottleneck.
- Pinned memory helps transfers but consumes a limited host resource.
- Calling `.item()`, printing CUDA tensors, or copying results to CPU can synchronize the GPU and distort timing.

For CUDA timing, synchronize around measurements or use CUDA events; device execution is asynchronous relative to Python.

10.5 Memory reduction techniques

- Mixed precision.
- Gradient checkpointing: recompute forward segments during backward, trading compute for activation memory.
- Smaller micro-batches with gradient accumulation.
- Efficient attention kernels.
- Freezing layers during transfer learning.
- Sharded training for parameters/gradients/optimizer state.
- Quantization for inference and, in specialized workflows, training.
- Avoid retaining graph-connected tensors in logs or Python containers.

10.6 Inference cost

Inference does not store gradients or optimizer state and normally needs fewer activations than training. Cost is influenced by batch size, sequence/image resolution, output generation length, model size, precision, hardware, preprocessing, and postprocessing.

Autoregressive LLM inference has a prefill phase over the prompt and a decode phase that produces tokens sequentially. Key-value caching avoids recomputing prior attention keys and values but consumes memory proportional to layers, sequence length, batch, and head dimensions.

11. Common Use Cases

1. **Image classification:** CNNs or Vision Transformers for defects, medical imaging, satellite imagery, and retail.
 2. **Object detection and segmentation:** bounding boxes, instance masks, and semantic maps for autonomous systems and inspection.
 3. **Natural-language processing:** classification, named-entity recognition, translation, summarization, and question answering.
 4. **LLM training and fine-tuning:** pretraining, supervised fine-tuning, preference optimization, LoRA/QLoRA integrations.
 5. **Generative vision:** GAN and diffusion-model training and sampling.
 6. **Speech/audio:** recognition, speaker identification, enhancement, and synthesis.
 7. **Recommendation:** embeddings, ranking networks, two-tower retrieval, and sequence recommenders.
 8. **Time series:** forecasting, anomaly detection, and representation learning.
 9. **Graph learning:** graph neural networks through ecosystem libraries.
 10. **Reinforcement learning:** policies, value functions, differentiable losses, and batched simulation data.
 11. **Scientific machine learning:** physics-informed losses, surrogate models, differentiable optimization.
 12. **Multimodal systems:** joint text-image/audio encoders, contrastive learning, and generative models.
-

12. Common Mistakes

Tensor and shape mistakes

- Confusing `[batch, classes]` with `[classes, batch]`.
- Accidentally removing the batch dimension with `x[0]`.
- Broadcasting incompatible intentions without raising an error.
- Flattening from dimension 0 and mixing the batch with features.
- Using `squeeze()` without a dimension and deleting a batch dimension when batch size is one.
- Passing NHWC images to layers expecting NCHW.
- Using `view` after `permute` without considering contiguity.

Dtype and device mistakes

- CPU inputs with CUDA model parameters.
- Float targets for `CrossEntropyLoss` class-index mode.
- Integer input passed to a floating-point linear layer.
- Creating a new CPU tensor inside a CUDA forward pass.
- Converting tensors requiring gradients to NumPy without detaching and moving to CPU.
- Using float16 where dynamic range is inadequate.

Autograd mistakes

- Forgetting to clear accumulated gradients.
- Calling `.item()`, `.detach()`, `torch.tensor(existing_tensor)`, or NumPy in the middle of a path that must remain differentiable.
- Updating parameters manually without no-grad semantics.

- Retaining graph-connected losses in a list, causing memory growth.
- Calling backward twice on a freed graph without a justified retained graph.
- Using in-place operations on values needed for backward.
- Expecting gradients on non-leaf intermediates without `retain_grad()`.

Model-state mistakes

- Calling `model.forward(x)` instead of `model(x)` and bypassing hooks/framework behavior.
- Storing layers in an ordinary Python list rather than `nn.ModuleList`, so parameters are not registered.
- Storing trainable tensors as plain tensors instead of `nn.Parameter`.
- Forgetting `model.eval()` during validation/inference.
- Assuming `eval()` disables gradients or freezes parameters.
- Creating new layers inside `forward`, reinitializing them each call and excluding them from the optimizer.
- Constructing the optimizer before replacing a classifier head, leaving new parameters out of optimizer groups.

Loss mistakes

- Applying softmax before `CrossEntropyLoss`.
- Applying sigmoid before `BCEWithLogitsLoss`.
- Using binary cross entropy for mutually exclusive multiclass labels without understanding the changed objective.
- Using accuracy for a heavily imbalanced safety-critical task.
- Averaging masked token loss over padding as well as real tokens.
- Shape mismatch that silently broadcasts predictions against targets.

Data and evaluation mistakes

- Fitting normalization, imputation, feature selection, or vocabulary on all data.
- Splitting correlated samples, such as frames or patient records, across sets.
- Applying random augmentation during validation.
- Using `shuffle=True` for a stateful time-series evaluation.
- Tuning hyperparameters or thresholds on the test set.
- Reporting only a favorable overall metric while subgroups fail.
- Ignoring class-to-index mapping when deploying.

Optimization mistakes

- Learning rate too large or too small.
- Forgetting `optimizer.step()` or placing it before backward.
- Scheduler step at the wrong frequency/order.
- Clipping scaled AMP gradients before unscaling.
- Applying weight decay indiscriminately without considering norms/biases in large models.
- Increasing model complexity before verifying the data and baseline.

Checkpoint and deployment mistakes

- Saving only weights but not preprocessing and label schema.

- Loading a checkpoint into an incompatible architecture and ignoring missing/unexpected keys.
 - Serializing entire objects and assuming portability across code changes.
 - Loading untrusted serialized objects.
 - Benchmarking without warm-up or CUDA synchronization.
 - Serving training-mode BatchNorm/Dropout.
-

13. Edge Cases / Limitations

Framework limitations and trade-offs

- Dynamic Python is easy to express but can add interpreter overhead for small operations.
- Compilation/export cannot always capture arbitrary Python, data-dependent side effects, or unsupported custom operations.
- Exact reproducibility across devices and versions is not guaranteed by seeding alone.
- GPU acceleration helps only when work is large enough to amortize launch and transfer overhead.
- Distributed training increases systems complexity and communication cost.

Data/model edge cases

- **Empty batch or dataset:** epoch denominators become zero; some layers reject empty dimensions.
- **Batch size one:** unqualified `squeeze()` can remove the batch dimension; BatchNorm training can be invalid or unstable.
- **Single class present:** some metrics become undefined and stratified splitting may fail.
- **Rare class:** a random split may place too few examples in validation/test.
- **Variable sequence length:** padding can dominate compute; masks must use the correct polarity/shape.
- **All-padding sequence:** softmax over fully masked positions can produce invalid values depending on implementation.
- **Zero-variance feature:** standardization divides by zero unless protected.
- **Very large logits:** naive exponentiation overflows; use stable framework losses.
- **Long sequences/high-resolution images:** activations or attention scores can exhaust memory.
- **Out-of-vocabulary/category IDs:** embeddings throw index errors or silently map incorrectly if preprocessing is inconsistent.

Autograd limitations

- Integer and Boolean operations are generally not differentiable.
- Discrete choices such as `argmax` block useful gradient flow.
- External NumPy or non-PyTorch code is invisible to autograd unless wrapped in a correctly implemented custom differentiation rule.
- Some operations have undefined or arbitrary subgradients at nondifferentiable points.
- Higher-order gradients greatly increase graph size and may expose unsupported backward paths.

Deployment limitations

- Python checkpoint compatibility depends on architecture code and versioning.
- Quantization may reduce accuracy, especially on sensitive layers or outlier-heavy distributions.
- Compiled/exported behavior must be validated against eager behavior on representative edge cases.

- Latency under concurrency can differ sharply from offline throughput benchmarks.
-

14. Variations

14.1 Eager PyTorch versus compiled PyTorch

- **What changes:** eager executes operations immediately; `torch.compile` attempts graph capture and optimization.
- **When to use:** eager for development/debugging; compile after profiling identifies meaningful overhead or fusion opportunity.
- **Importance:** placements—know the distinction; projects—benchmark; research—important for performance-sensitive models.

14.2 `nn.Sequential` versus custom `nn.Module`

- **What changes:** `Sequential` expresses a straight pipeline; custom `forward` supports branches, residuals, multiple inputs/outputs, and reused modules.
- **When to use:** use the simplest form matching the data flow.
- **Importance:** fundamental interview topic.

14.3 Manual training loop versus high-level trainer

- **What changes:** a manual loop exposes all control; trainer frameworks standardize logging, checkpointing, distributed execution, and callbacks.
- **When to use:** manual loops for learning, unusual research logic, and small projects; trainers when repeated engineering concerns justify them.
- **Importance:** placements expect the manual loop even if production uses a trainer.

14.4 Full fine-tuning versus frozen backbone

- **What changes:** full fine-tuning updates all pretrained parameters; feature extraction updates only a new head.
- **When to use:** frozen backbone for limited data/compute; full or partial unfreezing for domain shift and maximum quality.
- **Importance:** very important for projects and transfer-learning interviews.

14.5 FP32, FP16, BF16, and quantized inference

- **FP32:** stable baseline, larger memory.
- **FP16:** faster/smaller on suitable hardware but narrower dynamic range; commonly paired with scaling.
- **BF16:** wider exponent range and often easier training on supported hardware, with lower mantissa precision.
- **INT8/INT4:** primarily inference or specialized training workflows; needs calibration/quantization-aware methods and accuracy checks.
- **Importance:** essential for modern AI-engineering and LLM interviews.

14.6 Data parallel, sharded, tensor, and pipeline parallelism

Variation	What is split	Best fit
Data parallel/DDP	Samples; model replicated	Model fits on each device
Fully sharded	Model states across devices	Model state does not fit per device
Tensor parallel	Individual matrix/tensor operations	Very large layers
Pipeline parallel	Layer stages	Very deep large models

These can be combined for large-scale training. They are advanced placement topics and central to research/LLM infrastructure roles.

14.7 Dense versus sparse gradients

- **What changes:** dense parameters receive full gradient tensors; sparse embeddings may update only accessed rows.
- **When to use:** huge embedding tables with lookup-based access.
- **Importance:** important for recommendation and NLP infrastructure roles; optimizer support is restricted.

14.8 Custom autograd functions

Subclass `torch.autograd.Function` when a custom operation needs explicitly defined forward and backward behavior.

```
class DifferentiableClamp(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, minimum, maximum):
        return x.clamp(minimum, maximum)

    @staticmethod
    def backward(ctx, grad_output):
        # Straight-through estimator: deliberate surrogate gradient.
        return grad_output, None, None
```

This example does not use the true clamp derivative; it uses a straight-through estimator. Such surrogate gradients must be documented and experimentally justified. Validate custom derivatives with `torch.autograd.gradcheck` in double precision where applicable.

14.9 Gradient checkpointing

- **What changes:** selected activations are discarded during forward and recomputed during backward.
- **When to use:** activation memory is the bottleneck and extra compute is acceptable.
- **Importance:** high for large Transformers and diffusion models.

14.10 Distributed checkpoint and fault tolerance

- **What changes:** checkpoint state can be sharded and coordinated across ranks.

- **When to use:** models and optimizer states too large for one process or long expensive jobs requiring restart.
- **Importance:** advanced engineering/research infrastructure topic.

14.11 Quantization approaches

- **Dynamic quantization:** weights quantized and activations handled dynamically; simple for suitable linear-heavy models.
- **Post-training static quantization:** calibration data estimates activation ranges.
- **Quantization-aware training:** fake quantization during training adapts the model to quantization error.
- **When to use:** CPU/edge latency or memory constraints, subject to supported kernels and accuracy validation.

15. Related Topics

PyTorch versus NumPy

Both provide array/tensor operations and broadcasting. PyTorch adds accelerator devices, neural-network modules, automatic differentiation, distributed training, and deployment/performance tooling. NumPy remains excellent for CPU scientific preprocessing and interoperates with CPU tensors.

PyTorch versus TensorFlow/JAX

- PyTorch emphasizes a Pythonic eager model with autograd and optional compilation.
- TensorFlow provides its own eager/graph and production ecosystem choices.
- JAX centers on composable function transformations such as differentiation, vectorization, and compilation with a more functional style.

The best choice depends on team expertise, ecosystem, deployment target, and existing infrastructure. Interviews care more about sound tensor, autograd, optimization, and systems reasoning than framework tribalism.

Autograd versus backpropagation

Backpropagation is the reverse-mode chain-rule algorithm used for layered computations. Autograd is the software mechanism that records operations and applies differentiation rules; it can differentiate programs more general than a conventional neural-network stack.

Dataset versus DataLoader

The dataset defines how to access one sample. The loader defines batching, shuffling, sampling, collation, multiprocessing, and memory-transfer preparation.

Module versus Parameter

A module organizes computation and state. A parameter is one trainable tensor registered within a module. Modules can contain other modules recursively.

`train()` versus `eval()` versus `no_grad()`

- `train()` enables training behavior for stateful/stochastic layers.
- `eval()` enables inference behavior for those layers.
- `no_grad()` disables graph recording.
- `inference_mode()` is a stronger pure-inference context.

Correct validation normally needs `eval()` plus `inference_mode()`.

Cross entropy versus BCE

Cross entropy models one mutually exclusive class distribution. Binary cross entropy models independent Bernoulli outputs and is therefore suitable for binary or multilabel tasks. Output and target shapes differ.

Adam versus AdamW

Adam combines adaptive moment estimates with a gradient update. AdamW decouples weight decay from that adaptive update, giving decay behavior closer to direct parameter shrinkage.

BatchNorm versus LayerNorm

BatchNorm uses statistics across examples/spatial positions per channel and changes behavior between train and eval. LayerNorm normalizes features within each example and does not need running batch statistics, making it natural for variable-length sequence/Transformer models.

DDP versus `DataParallel`

DDP uses multiple processes and efficient gradient synchronization and is the standard multi-GPU training approach. `DataParallel` uses one process with scatter/gather overhead and a central device bottleneck; it is simpler but generally less scalable.

Saving versus exporting

Saving a state dict preserves parameter/buffer values for reconstruction in PyTorch. Exporting produces a deployable program representation with stricter constraints and a runtime contract.

PyTorch and Hugging Face

Hugging Face libraries provide pretrained models, tokenizers, datasets, and trainers, while PyTorch supplies tensor operations, modules, autograd, and optimization underneath many workflows. Understanding native PyTorch makes customization and debugging far easier.

PyTorch and MLOps

PyTorch trains and runs models; MLOps covers data/model versioning, experiment tracking, CI, deployment, monitoring, rollback, governance, and reproducibility. A `.pt` file alone is not a production ML system.

16. Interview Questions

1. What is PyTorch?

PyTorch is a tensor-computation and automatic-differentiation framework. It provides accelerator-backed tensors, dynamic computation graphs, neural-network modules, optimizers, data utilities, and distributed/performance tools.

2. What is the difference between a tensor and `nn.Parameter`?

Both hold tensor data. Assigning an `nn.Parameter` as a module attribute registers it as trainable model state, so it appears in `parameters()` and `state_dict()`. A plain tensor does not automatically do so.

3. What happens when `loss.backward()` is called?

Autograd traverses the recorded graph backward and computes vector-Jacobian products using each operation's derivative. For leaf tensors requiring gradients, results accumulate into `.grad`.

4. Why must gradients be zeroed?

PyTorch accumulates gradients by addition. Without clearing them, each update uses current plus previous gradients. This is useful for deliberate accumulation but wrong in the ordinary one-batch-per-step loop.

5. Why use `optimizer.zero_grad(set_to_none=True)`?

Setting gradients to `None` can avoid memory writes and clearly indicates parameters that received no gradient. On the next backward, a fresh gradient is allocated. Optimizers may distinguish `None` from an explicit zero gradient.

6. What is a dynamic computation graph?

The graph is constructed as tensor operations execute, so Python branches, loops, and model structure can change between calls. Backward differentiates the graph actually executed.

7. What is a leaf tensor?

A leaf tensor is typically user-created rather than produced by a tracked operation. Model parameters are leaves. Autograd retains `.grad` on leaves by default; non-leaf intermediates require `retain_grad()` if their gradients need inspection.

8. What is the difference between `detach()`, `no_grad()`, and `inference_mode()`?

`detach()` disconnects a particular tensor from its current graph. `no_grad()` prevents recording operations in a region. `inference_mode()` is a stronger, lower-overhead inference context with additional restrictions on autograd metadata.

9. Why do we call `model.eval()`?

It switches modules such as Dropout and BatchNorm to evaluation behavior. It does not disable gradient calculation, so inference should also use `inference_mode()` or `no_grad()`.

10. Why should `CrossEntropyLoss` receive logits rather than softmax probabilities?

It combines log-softmax and negative log-likelihood in a numerically stable implementation. Applying softmax first duplicates work, reduces numerical stability, and changes the expected input.

11. What shapes and dtypes does multiclass cross entropy expect?

For ordinary classification, logits have shape `[N, C]` and floating dtype; targets have shape `[N]`, `torch.long` dtype, and class IDs from `0` through `C-1`. Higher-dimensional variants support spatial targets.

12. What is the difference between binary and multilabel classification?

Binary classification has one yes/no target. Multilabel classification has several independent yes/no labels per sample. Both commonly use logits and `BCEWithLogitsLoss`, but multilabel output/target shape is `[N, C]` and each label receives an independent sigmoid.

13. What does `requires_grad=True` mean?

It tells autograd to record differentiable operations needed to compute derivatives with respect to that tensor or upstream leaves. Only floating/complex tensors support ordinary gradients.

14. Why can `.item()` break a computation graph?

`.item()` converts a one-element tensor to a Python scalar, which has no autograd history. It is appropriate for logging, not for values that must remain in differentiable computation.

15. What is broadcasting, and what bug can it cause?

Broadcasting virtually expands compatible size-one or missing dimensions. It can silently create unintended pairwise operations—for example, adding `[B, 1]` to `[B]` yields `[B, B]`.

16. What is a contiguous tensor?

It has a memory layout whose strides follow the expected contiguous ordering. Transposes and slices can make non-contiguous views. Many operations support them; `contiguous()` copies only when a required layout is absent.

17. What is the difference between `view` and `reshape`?

`view` requires a stride-compatible view. `reshape` returns a view when possible but may allocate a copy. Code must not rely on `reshape` sharing storage.

18. Why use `ModuleList` instead of a Python list?

`ModuleList` registers child modules. Modules stored only in a plain list are invisible to recursive parameter traversal, device movement, mode switching, and state dictionaries.

19. What is a buffer?

A buffer is non-parameter tensor state registered with a module, such as BatchNorm running statistics. It moves with `.to(device)` and can be saved, but is not updated by an optimizer.

20. How do you freeze and unfreeze a model?

Set `requires_grad_(False)` on frozen parameters and construct the optimizer from parameters intended for update. On unfreezing, set it true and ensure the optimizer contains those parameters, often with a smaller

learning rate.

21. Why can a model have no gradients even though backward ran?

Possible causes include detached tensors, global no-grad context, parameters not used in forward, layers stored unregistered, nondifferentiable operations such as argmax, or the optimizer/model referencing different parameter objects.

22. What causes exploding or vanishing gradients?

Repeated Jacobian multiplication can make gradient norms grow or shrink exponentially. Initialization, normalization, residual connections, gated recurrence, suitable activations, learning-rate control, and clipping help.

23. Explain mixed-precision training.

Autocast selects lower precision for suitable operations to improve throughput and memory. Sensitive operations remain safer. With float16, gradient scaling reduces underflow; the scaler skips an update when non-finite gradients are detected and adjusts its scale.

24. Why must gradients be unscaled before clipping under AMP?

Clipping scaled gradients measures the artificial scale rather than their true norm. `scaler.unscale_(optimizer)` restores their intended scale before clipping.

25. What does a `DataLoader` do?

It takes samples from a dataset according to a sampler, combines them using collation, forms batches, and optionally loads them in worker processes with pinned-memory support.

26. Why can increasing `num_workers` make loading slower?

Worker startup, interprocess communication, serialization, storage contention, small datasets, heavy thread use inside each worker, or limited CPU/memory can exceed the saved loading time. It must be benchmarked.

27. How do you prevent data leakage in a PyTorch pipeline?

Create splits at the correct independent unit first. Fit scalers, imputers, vocabularies, feature selectors, and threshold/calibration logic only from training or designated validation data. Keep test untouched until final evaluation.

28. What should a training checkpoint include?

For inference: model state, architecture/config, preprocessing, label mapping, and version metadata. For resume: also optimizer, scheduler, AMP scaler, epoch/step, best score, and sometimes RNG and sampler state.

29. Why prefer a `state_dict` over saving an entire model object?

A state dict is an explicit mapping of tensor state and is less coupled to Python object pickling and exact module import paths. The architecture must still be reconstructed consistently.

30. Explain DDP at a high level.

Each process owns one model replica and a distinct data shard. Forward and backward run locally; gradient buckets are synchronized across processes, typically by all-reduce, so replicas apply equivalent updates.

31. Why call `DistributedSampler.set_epoch(epoch)`?

It changes the deterministic shuffle seed per epoch consistently across ranks. Without it, every epoch may use the same sample order.

32. Why is DDP usually preferred over `DataParallel`?

DDP uses one process per device, avoids a central Python/GIL bottleneck, distributes gradient communication efficiently, and scales better. `DataParallel` scatters/gathers through one process and a primary device.

33. What is gradient checkpointing?

It stores fewer intermediate activations and recomputes selected forward segments during backward. It lowers activation memory at the cost of extra compute.

34. How do you debug a model stuck at chance accuracy?

Check input/target/output contracts, label distribution and mapping, loss choice, model registration, optimizer parameters, gradient presence and magnitude, learning rate, modes, and preprocessing. Then try to overfit a tiny batch.

35. How do you benchmark GPU inference correctly?

Use representative shapes and preprocessing, evaluation plus inference mode, warm up compilation/kernels, synchronize or use CUDA events, measure multiple iterations and tail latency, and include transfer/postprocessing costs required by the real service.

36. When would you write a custom `autograd.Function`?

When implementing an operation whose derivative PyTorch cannot infer, integrating a custom kernel, or intentionally defining a surrogate gradient. Validate forward and backward carefully, including `gradcheck` where suitable.

37. What is an in-place autograd error?

Backward may need a saved tensor value, but an in-place operation changed it after forward. PyTorch tracks tensor versions and raises rather than compute a silently incorrect gradient.

38. Why might training and inference predictions differ?

Dropout, BatchNorm statistics, stochastic preprocessing, precision, preprocessing mismatch, nondeterministic kernels, hidden state, checkpoint mismatch, or a missing `eval()` call can cause differences.

39. What does `torch.compile` do, and when can it fail to help?

It captures and optimizes executable graph regions, potentially fusing operations and reducing Python overhead. Frequent graph breaks, dynamic behavior, unsupported operations, compile overhead, or an already kernel-bound workload can limit gains.

40. Loss decreased, but business performance did not. Why?

The surrogate loss may not align with the business objective, data may not match production, thresholds may be wrong, probabilities may be uncalibrated, aggregate metrics may hide subgroup failures, or offline features may leak unavailable information.

17. Practice Tasks

Task 1: Small coding task—linear regression from scratch

Implement linear regression using only tensors and autograd:

- Generate $y=3x-2+\epsilon$.
- Create scalar w and b with gradients.
- Compute MSE manually.
- Update values inside `torch.no_grad()`.
- Clear gradients after every update.
- Compare learned values with 3 and -2.

Extension: derive the gradients by hand and compare them to autograd.

Task 2: Dataset-based project—image classification

Train a CNN on Fashion-MNIST or CIFAR-10:

- Create train/validation/test partitions.
- Apply augmentation only to training.
- Report loss, accuracy, macro F1, and confusion matrix.
- Compare an MLP with a CNN.
- Save the best checkpoint and a prediction script.

Task 3: Experiment—optimizer and learning-rate comparison

Using a fixed model and split, compare:

- SGD with momentum.
- Adam.
- AdamW.
- Constant learning rate versus cosine schedule.

Keep the number of optimizer steps constant. Plot train/validation loss against steps, record best score, time, and memory, and explain whether improvements are optimization speed or generalization.

Task 4: Debugging task—broken classifier

Find and fix every bug:

```
model.train()
for x, y in validation_loader:
    probabilities = torch.softmax(model(x.cuda()), dim=1)
    loss = nn.CrossEntropyLoss()(probabilities, y.float())
    loss.backward()
```

Expected findings:

- Validation should use `eval()`.
- Use `inference_mode()` and no backward.
- Model/device placement must match inputs and targets.
- `CrossEntropyLoss` should receive logits.
- Targets should be class-index `long` tensors.
- Metrics should be aggregated across the full validation set.

Task 5: Analysis task—tiny-batch overfit

Take 16–64 examples and train until nearly zero loss. If the model cannot memorize them, inspect:

- Label/input alignment.
- Parameter registration.
- Gradient flow.
- Output/target shape.
- Loss contract.
- Learning rate.
- Excessive regularization/augmentation.

Write a one-page diagnosis based on measured evidence.

Task 6: Extension—mixed precision and compilation

Add AMP and optional `torch.compile` to an existing model. Benchmark:

- Peak accelerator memory.
- Examples per second.
- Validation metric.
- First-call latency and warmed-up latency.

Do not claim speedup unless end-to-end measurements support it.

Task 7: Custom dataset and collation

Build a text dataset returning variable-length token ID tensors. Write a `collate_fn` that pads each batch, returns lengths or an attention mask, and ensures padding tokens do not contribute to the loss.

Task 8: Checkpoint resume

Train for several epochs, save model/optimizer/scheduler/scaler state, restart the process, and resume. Verify that the learning rate, global step, and validation trajectory continue correctly.

Task 9: Transfer learning

Fine-tune a pretrained image model in two phases:

1. Freeze the backbone and train a new head.
2. Unfreeze the final block with a smaller learning rate.

Compare quality, training time, and overfitting.

Task 10: Distributed reasoning exercise

For 4 GPUs, per-GPU batch 32, and accumulation 2:

- Compute effective batch size: $(4 \times 32 \times 2 = 256)$.
 - Explain how the distributed sampler avoids duplication.
 - Explain when gradients synchronize.
 - Describe how to aggregate global accuracy.
-

18. Project Ideas

Project 1: Production-Ready Visual Defect Classifier

What it does: Classifies manufacturing images into defect categories, provides confidence scores, and flags uncertain samples for review.

Tech stack: PyTorch, torchvision, FastAPI, Docker, MLflow or another experiment tracker, and a lightweight dashboard.

Dataset suggestion: NEU surface defect dataset, MVTec AD for anomaly-oriented extension, or a carefully documented custom dataset.

Implementation depth:

- Transfer learning with a pretrained CNN/ViT.
- Group-aware splits by production batch or source.
- Augmentation ablation.
- Macro F1 and per-class recall.
- Calibration and abstention threshold.
- Exported inference endpoint with versioned preprocessing.
- Latency and failure monitoring.

Resume value: Demonstrates computer vision, transfer learning, evaluation under imbalance, deployment, and production thinking—not just notebook accuracy.

Project 2: Semantic Search and Reranking System

What it does: Retrieves relevant documents with learned embeddings and optionally reranks top candidates using a cross-encoder.

Tech stack: PyTorch, Hugging Face Transformers, FAISS or a vector database, FastAPI, evaluation scripts, Docker.

Dataset suggestion: MS MARCO subsets, BEIR datasets, Natural Questions, or a domain-specific question-document corpus.

Implementation depth:

- Fine-tune a bi-encoder with contrastive loss.
- Build an embedding index.
- Measure Recall@K, MRR, and NDCG.
- Add hard-negative mining.
- Compare embedding-only retrieval with reranking.
- Measure quality-latency trade-offs.

Resume value: Connects PyTorch training to NLP, vector retrieval, RAG foundations, offline evaluation, and service design.

Project 3: Time-Series Anomaly Detection Platform

What it does: Learns normal multivariate sensor behavior and flags abnormal windows with explanations based on reconstruction/prediction error.

Tech stack: PyTorch, Pandas, scikit-learn for preprocessing/baselines, FastAPI or batch inference, Plotly/dashboard tooling, Docker.

Dataset suggestion: NASA turbofan, SWaT/WADI where licensing permits, UCR anomaly archive, or public server-metric datasets.

Implementation depth:

- Chronological splits with no future leakage.
- MLP/LSTM/Transformer or autoencoder comparison.
- Threshold selection on validation data.
- Event-level precision/recall and detection delay.
- Drift monitoring and backtesting.
- Reproducible checkpoint and scaler storage.

Resume value: Shows sequence modeling, leakage-aware validation, rare-event evaluation, thresholding, and end-to-end ML engineering.

19. Quick Revision

Key idea

PyTorch represents data and parameters as tensors, records differentiable forward operations in a dynamic graph, computes gradients through reverse-mode autograd, and updates registered parameters through optimizers.

Main formulas

Linear layer:

$$Z = XW^T + b$$

Gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Multiclass cross entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_i \log \frac{e^{z_{i,y_i}}}{\sum_c e^{z_{i,c}}}$$

Standard training pattern

```
model.train()
for x, y in train_loader:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad(set_to_none=True)
    loss = criterion(model(x), y)
    loss.backward()
    optimizer.step()
```

Standard evaluation pattern

```
model.eval()
with torch.inference_mode():
    for x, y in validation_loader:
        logits = model(x.to(device))
```

When to use

- Custom deep-learning research and rapid experiments.
- GPU-accelerated model training.
- Vision, NLP, audio, recommendation, generative AI, RL, and scientific ML.
- Production inference when its ecosystem and deployment path fit the system.

Important metrics

- Classification: accuracy, precision, recall, F1, ROC-AUC/PR-AUC, log loss, calibration.
- Regression: MAE, RMSE, (R^2), domain cost.
- Ranking: Recall@K, MRR, NDCG.
- Segmentation/detection: IoU, Dice, mAP.
- Systems: latency percentiles, throughput, peak memory, cost, error rate.

Common traps

- Softmax before `CrossEntropyLoss`.
- Sigmoid before `BCEWithLogitsLoss`.
- Forgetting gradient clearing.

- Forgetting `eval()` plus inference context.
- Device/dtype/shape mismatch.
- Unregistered modules/parameters.
- Leakage during split or preprocessing.
- Incorrect epoch loss aggregation.
- Saving weights without preprocessing metadata.
- Optimizing test performance through repeated test access.

Interview one-liner

A PyTorch training step clears accumulated gradients, runs a forward pass to build the autograd graph, computes a scalar loss, backpropagates vector-Jacobian products into parameter gradients, and lets an optimizer update registered parameters.

20. Final Cheat Sheet

Item	PyTorch answer
Definition	Tensor, autograd, and deep-learning framework with accelerator and distributed support
Input	Tensors with deliberate shape, dtype, device, and preprocessing
Output	Predictions such as values, logits, embeddings, masks, boxes, or token scores
Model	<code>nn.Module</code> containing child modules, <code>nn.Parameters</code> , and buffers
Graph	Built from executed differentiable tensor operations; traversed backward by autograd
Main training steps	Load → device transfer → zero grad → forward → loss → backward → optimizer step
Evaluation steps	<code>eval()</code> → <code>inference_mode()</code> → forward → aggregate metrics
Multiclass contract	<code>[N,C]</code> logits + <code>[N]</code> long targets + <code>CrossEntropyLoss</code>
Binary/multilabel contract	Logits and float 0/1 targets of matching shape + <code>BCEWithLogitsLoss</code>
Key hyperparameters	Learning rate, batch size, optimizer, weight decay, architecture, dropout, scheduler, epochs
Common optimizers	SGD/momentum, Adam, AdamW
Performance tools	AMP, compilation, efficient loaders, gradient checkpointing, DDP/sharding, quantization
Save for inference	State dict, architecture config, preprocessing, labels, thresholds, versions
Save for resume	Inference state plus optimizer, scheduler, scaler, epoch/step, RNG as needed
Classification metrics	Accuracy, precision/recall/F1, PR-AUC/ROC-AUC, confusion matrix, calibration

Item	PyTorch answer
Pros	Pythonic, flexible, strong autograd, large ecosystem, research-to-production path
Cons	Easy to write silent shape/leakage errors; GPU/distributed/export performance needs systems care
Best use cases	Custom neural networks, research, vision/NLP/LLMs, GPU training, differentiable programs

Ten commands to remember

```

model.to(device)           # move registered model state
model.train()              # training behavior
model.eval()               # evaluation behavior
optimizer.zero_grad(set_to_none=True) # clear accumulated gradients
logits = model(inputs)     # call module, not forward directly
loss = criterion(logits, targets) # scalar objective
loss.backward()            # compute/accumulate gradients
optimizer.step()           # update parameters
torch.save(model.state_dict(), "model.pt") # save tensor state
with torch.inference_mode(): ... # low-overhead inference

```

Final mental model

```

Dataset → DataLoader → batch tensors → model forward → logits/predictions
                                     ↓
targets —————→ loss
                                     ↓ backward
                                parameter gradients
                                     ↓ optimizer
                                updated parameters

```

If an interview problem is confusing, return to four contracts:

1. **Data contract:** What do one sample and one batch contain?
2. **Shape contract:** What are the dimensions before and after every major layer?
3. **Objective contract:** Do output, target, and loss agree?
4. **State contract:** Which values are parameters, buffers, optimizer state, preprocessing state, and evaluation-only metrics?

Those four checks explain and prevent a large fraction of real PyTorch failures.